CS 305 Project One

Document Revision History

Version	Date	Author	Comments
1.0	9/19/2026	Grace Jungclas	

Client



Developer
Grace Jungclas, MA, AT Ret

1. Interpreting Client Needs
Artemis Financial provides customers with tailored financial plans covering savings, retirement, investments, and insurance. Secure financial communications matter because the Center for Internet Security (CIS) reports that the average cost of a financial security breach is $ 5. 9 million, second only to the healthcare sector (Center for Internet Security [CIS], 2024). Attackers are driven by financial gain and can achieve their objectives through system intrusions, misconfigurations, and social engineering. The number of threats facing financial organizations has increased because they have a relatively large attack surface, driven by outdated IT infrastructure, open sharing of financial data, limited cybersecurity awareness and expertise, and rising fraud costs. That is why financial institutions are among the most heavily regulated businesses (CIS, 2024). For example, the Financial Modernization Act of 1999 (a.k.a. the Gramm-Leach-Bliley Act) instituted the Financial Privacy Rule and the Safeguards Rule, which restrict how financial institutions communicate customers' information and require those institutions to maintain those safeguards, respectively. While it is unclear whether the company engages internationally, its public- facing RESTful application programming interface (API) routes traffic across global networks, requiring defensive coding and robust encryption to protect against potential worldwide attack vectors. To protect against current and near- future external threats, the company can leverage the CIS Community Defense Model (CDM 2. 2.0), which applies Implementation Group 1 (IG 1) essential cyber hygiene safeguards against the top five attack types: Malware, Ransomware, Web Application Hacking, Insider and Privilege Misuse, and Targeted Intrusions. Because web applications account for over 80% of breaches, modernization requires active defenses against injection vulnerabilities and exploitation of public- facing endpoints, such as constraining API endpoints with explicit `@GetMapping` declarations rather than unrestricted `@RequestMapping` mappings to prevent HTTP method tampering. Modern application architectures also rely heavily on open- source libraries and evolving web frameworks. While these third-party dependencies accelerate development, they introduce significant supply-chain risks when unpatched, as shown by outdated artifacts such as Spring Framework, Apache Tomcat, and FasterXML Jackson Databind. Modernization requirements must therefore incorporate automated Software Composition Analysis (SCA), such as the OWASP Dependency- Check Maven plugin, into the build lifecycle to continuously detect, evaluate, and patch known Common Vulnerabilities and Exposures (CVEs) before deployment (Center for Internet Security, 2022).

2. Areas of Security
- Input Validation (Secure Input and Representations): Since Artemis Financial wishes to have a web application that accepts requests from external sources, it is necessary for the application to impose strict limitations on all parameters regarding their type, format, and range. If rigorous input validation and sanitization are not carried out at the boundary, data supplied by clients, which should not be trusted, could spread to downstream components, thus providing opportunities for injection attacks (for example, SQL injection or expression language injection) and leading to unexpected program behavior.
- APIs (Secure API Interactions): The main interface is a public RESTful API. With regard to security, the endpoints strictly adhere to the principle of least privilege by linking themselves solely to the appropriate HTTP methods (for example, using `@GetMapping` rather than `@RequestMapping` without constraints, which would allow any HTTP verbs), by enforcing authorization at the method level, and by preventing tampering with HTTP verbs.

- Cryptography (Encryption Use and Vulnerabilities): To protect financial planning information and data on customers' assets it is necessary to use strong encryption when data is being transmitted and when it is stored. Cryptography is essential for setting up TLS/HTTPS communication channels, for managing secure digital certificates and keystores, and for using cryptographic hash functions to check that data integrity is maintained during transfers.
- Client/Server (Secure Distributed Computing): The application operates in a distributed network model where remote clients interact with centralized server endpoints. Securing this boundary requires validating session state, limiting unauthorized exposure of remote services, preventing man-in-the-middle (MitM) interception, and ensuring that external clients cannot exploit architectural trust assumptions across distributed tiers.
- Code Error (Secure Error Handling): Poor error handling may accidentally cause internal architecture, fragments of the database schema, or detailed framework stack traces to be made available to external parties. The application should use standardized, centralized exception handling to log useful diagnostic information internally and provide users with sanitized, generic status messages.
- Code Quality (Secure Coding Practices and Patterns): This area assesses the maintainability, reliability, and defensive capabilities of the codebase. It involves avoiding anti-patterns, getting rid of dead or insecure code elements, enforcing strict compiler and linter standards, and ensuring proper open-source dependency hygiene by identifying and updating libraries that have known CVEs.
- Encapsulation (Secure Data Structures): This does not apply directly, since it is not a main factor in the immediate vulnerability profile.

3. Manual Review

1. Unrestricted HTTP Method Exposure
    a. Security Area: API
    b. File & Location: `CRUDController.java` (Method: `CRUD`)
    c. Description & Impact: The read endpoint uses `@RequestMapping("/read")` without specifying an explicit HTTP request method constraint (e.g., `RequestMethod.GET`). Omitting the method parameter, in Spring MVC, binds the endpoint to all valid HTTP verbs (`GET`, `POST`, `PUT`, `DELETE`, `PATCH`, `HEAD`, `OPTIONS`). This expands the attack surface beyond read-only intent, violates the principle of least privilege, and leaves the application susceptible to HTTP method tampering.
2. Hardcoded Database Credentials in Source Code
    a. Security Area: Cryptography
    b. File & Location: `DocData.java` (Method: `read_document`)
    c. Description & Impact: Database connection parameters and authentication secrets are hardcoded in plaintext directly into the source code. Anyone with access to the source code, version control history, or compiled `.class` files can immediately extract administrative database credentials (`root`), leading to unauthorized data access and total database compromise.
3. Cleartext Database Communication Channel
    a. Security Area: Cryptography / Client-Server
    b. File & Location: `DocData.java` (Method: `read_document`)

   c. Description & Impact: The JDBC connection URL (`jdbc:mysql://localhost:3306/test`) does not enforce Transport Layer Security (TLS/SSL). Network traffic between the application service and the MySQL database instance is transmitted in plaintext, exposing sensitive financial queries, result sets, and credentials to packet sniffing and Man-in-the-Middle (MitM) attacks across the network segment.

4. Diagnostic Information Disclosure
   a. Security Area: Code Error (Secure Error Handling)
   b. File & Location: `DocData.java` (Method: `read_document`)
   c. Description & Impact: The exception-handling block catches `SQLException` and outputs diagnostic trace information directly to standard error using `e.printStackTrace()`. Printing unhandled stack traces in a live production runtime or containerized environment exposes internal system paths, framework versions, database drivers, and schema implementation details, providing external threat actors with technical reconnaissance data.
5. Unmanaged Database Sockets and Connection Leak
   a. Security Area: Code Quality / Error Handling
   b. File & Location: `DocData.java` (Method: `read_document`)
   c. Description & Impact: The `Connection` instance `con` is created within a try block but is neither closed in a finally block nor managed with a try-with-resources construct. Each successful invocation of `read_document` that establishes a connection leaves the underlying TCP socket open. Repeated calls exhaust sockets and the connection pool, resulting in an application-level denial of service (DoS).
6. Unvalidated Parameter Reflection
   a. Security Area: Input Validation
   b. File & Location: `GreetingController.java` (Method: `greeting`)
   c. Description & Impact: The controller received untrusted client input through `@RequestParam(value = "name", defaultValue = "World") String name` and directly formats it into the response object via `String.format(template, name)` without validation, length limits, or sanitization. If downstream web clients or administrative frontends render this payload without contextual output encoding, malicious scripts can execute in the user's browser session.
7. Broken Encapsulation and Package-Private Data Access
   a. Security Area: Encapsulation / Code Quality
   b. File & Location: `customer.java`
   c. Description & Impact: The `account_balance` variable is declared without an access modifier (`int account_balance;`), defaulting to package-private visibility. Any class in `com.twk.restservice` can directly inspect and modify account balance values without passing through business validation methods. Furthermore, `showInfo()` exposes the raw internal `account_number` without permission checks, violating data-hiding standards and role-based access control.
8. Unbounded Numeric Input and Missing Business Logic Validation
   a. Security Area: Input Validation / Code Quality
   b. File & Location: `customer.java` (Method: `deposit`)
   c. Description & Impact: The `deposit(int a)` mutator accepts a raw signed primitive integer and adds it directly to account_balance without range or sign validation

(`account_balance = account_balance + a;`). Passing negative integers allows callers to reduce the balance without authorization. Additionally, passing values near `Integer.MAX_VALUE` can trigger integer overflow, turning positive financial balances negative without warning.

4. Static Testing

Severity: CRITICAL

1. **Dependency:** tomcat-embed-websocket-9.0.30.jar
    a. **CVE ID:** CVE-2020-1938 ("Ghostcat")
    b. **Description:** When using the Apache JServ Protocol (AJP), care must be taken when trusting incoming connections to Apache Tomcat. Tomcat treats AJP connections as having higher trust than, for example, a similar HTTP connection. If such connections are available to an attacker, they can be exploited in ways that may be surprising. In Apache Tomcat 9.0.0.M1 to 9.0.0.30, 8.5.0 to 8.5.50 and 7.0.0 to 7.0.99, Tomcat shipped with an AJP Connector enabled by default that listened on all configured IP addresses. It was expected (and recommended in the security guide) that this Connector would be disabled if not required. This vulnerability report identified a mechanism that allowed: - returning arbitrary files from anywhere in the web application - processing any file in the web application as a JSP Further, if the web application allowed file upload and stored those files within the web application (or the attacker was able to control the content of the web application by some other means) then this, along with the ability to process a file as a JSP, made remote code execution possible.
    c. **Solution:** It is important to note that mitigation is only required if an AJP port is accessible to untrusted users. Users wishing to take a defence-in-depth approach and block the vector that permits returning arbitrary files and execution as JSP may upgrade to Apache Tomcat 9.0.31, 8.5.51 or 7.0.100 or later. A number of changes were made to the default AJP Connector configuration in 9.0.31 to harden the default configuration. It is likely that users upgrading to 9.0.31, 8.5.51 or 7.0.100 or later will need to make small changes to their configurations.
    d. **Attribution documenting how the vulnerability has been identified or documented:**
        i. **Source:** Discovered by Chaitin Tech and reported by Apache Software Foundation
        ii. **NVD Publish Date:** Feb 24, 2020
2. **Dependency:** spring-web-5.2.3.RELEASE.jar; spring-core-5.2.3.RELEASE.jar; spring-webmvc-5.2.3.RELEASE.jar
    a. **CVE ID:** CVE-2022-22965 ("Spring4Shell")
    b. **Description:** Spring MVC or Spring WebFlux application running on JDK 9+ may be vulnerable to remote code execution (RCE) via data binding.
    c. **Solution:** Apply updates per vendor instructions
    d. **Attribution documenting how the vulnerability has been identified or documented:**
        i. **Source:** codeplutos and meizjm3i of AntGroup FG Security Lab, coordinated through VMware/ Spring
        ii. **NVD Publish Date:** Apr 01, 2022
3. **Dependency:** spring-boot-starter-web-2.2.4.RELEASE.jar; spring-boot-2.2.4.RELEASE.jar
    a. **CVE ID:** CVE-2023-20873

b. **Description:** In Spring Boot versions 3.0.0 - 3.0.5, 2.7.0 - 2.7.10, and older unsupported versions, an application that is deployed to Cloud Foundry could be susceptible to a security bypass.
c. **Solution:** Users of affected versions should apply the following mitigation: 3.0.x users should upgrade to 3.0.6+. 2.7.x users should upgrade to 2.7.11+. Users of older, unsupported versions should upgrade to 3.0.6+ or 2.7.11+.
d. **Attribution documenting how the vulnerability has been identified or documented:**
    i. **Source:** VMware
    ii. **NVD Publish Date:** April 20, 2023

4. **Dependency:** snakeyaml-1.25.jar
    a. **CVE ID:** CVE-2022-1471
    b. **Description:** SnakeYaml's Constructor() class does not restrict types which can be instantiated during deserialization. Deserializing yaml content provided by an attacker can lead to remote code execution.
    c. **Solution:** Use SnakeYAML's SafeConstructor when parsing untrusted content to restrict deserialization, and upgrade to version 2.0 and beyond.
    d. **Attribution documenting how the vulnerability has been identified or documented:**
        i. **Source:** Zhihong Tian of Google Open Source Security Team (GOSST)
        ii. **NVD Publish Date:** Dec 01, 2022

Severity: High

1. **Dependency:** logback-core-1.2.3.jar
    a. **CVE ID:** CVE-2023-6378
    b. **Description:** A serialization vulnerability in logback receiver component part of logback version 1.4.11 allows an attacker to mount a Denial-Of-Service attack by sending poisoned data.
    c. **Solution:** Upgrade ch.qos.logback:logback-core to 1.2.13, 1.2.12, or 1.2.12+. As an immediate mitigation, disable remote receiver components (e.g., ServerSocketReceiver) if serialized socket logging is not required.
    d. **Attribution documenting how the vulnerability has been identified or documented:**
        i. **Source:** Switzerland Government Common Vulnerability Program
        ii. **Credit:** Yakov Shafranovich, Amazon Web Services
        iii. **NVD Publish Date:** Nov 29, 2023
2. **Dependency:** jackson-databind-2.10.2.jar
    a. **CVE ID:** CVE-2020-25649
    b. **Description:** A flaw was found in FasterXML Jackson Databind, where it did not secure entity expansion properly. This flaw exposes the application to XML external entity (XXE) attacks. The highest threat from this vulnerability is data integrity.
    c. **Solution:** Upgrade jackson-databind to version 2.10.5.1, 2.11.4, or 2.12.0 or later. Alternatively, upgrade the parent spring-boot-starter-parent dependency to version 2.3.6.RELEASE or higher to automatically pull in the patched transitive library. If an immediate binary upgrade cannot be performed, disable external entity resolution and document-type definitions (SUPPORT_DTD = false) on any underlying XML parser components interacting with the Jackson XML extension
    d. **Attribution documenting how the vulnerability has been identified or documented:**
        i. **Source:** Mike Damm coordinated through Red Hat, Inc. and FasterXML

   ii. **NVD Publish Date:** Dec 03, 2020

3. **Dependency:** bcprov-jdk15on-1.46.jar
   a. **CVE ID:** CVE-2016-1000338
   b. **Description:** In Bouncy Castle JCE Provider version 1.55 and earlier, the DSA does not fully validate ASN.1 encoding of the signature on verification. It is possible to inject extra elements in the sequence making up the signature and still have it validate, which in some cases may allow the introduction of 'invisible' data into a signed structure.
   c. **Solution:** Upgrade the Bouncy Castle JCE Provider (bcprov-jdk15on) to version 1.56 or higher (or migrate to the current standard artifact bcprov-jdk18on). If an immediate library upgrade is not possible, avoid the affected Bouncy Castle DSA signature verification provider and configure applications to rely exclusively on Java's built-in cryptographic providers (such as the default SUN security provider) for validating ASN.1-encoded DSA signatures.
   d. **Attribution documenting how the vulnerability has been identified or documented:**
      i. **Source:** MITRE
      ii. **NVD Publish Date:** Jun 01, 2018

Severity: Medium

1. **Dependency:** hibernate-validator-6.0.18.Final.jar
   a. **CVE ID:** CVE-2025-35036
   b. **Description:** Hibernate Validator before 6.2.0 and 7.0.0, by default and depending on how it is used, may interpolate user-supplied input in a constraint violation message with Expression Language. This could allow an attacker to access sensitive information or execute arbitrary Java code. Hibernate Validator as of 6.2.0 and 7.0.0 no longer interpolates custom constraint violation messages with Expression Language and strongly recommends not allowing user-supplied input in constraint violation messages. CVE-2020-5245 and CVE-2025-4428 are examples of related, downstream vulnerabilities involving Expression Language interpolation of user-supplied data.
   c. **Solution:** Upgrade Hibernate Validator to version 6.2.0.Final, 7.0.0.Final, or higher. If an immediate upgrade isn't feasible, implement defensive code patterns by disabling default Expression Language interpolation in custom ConstraintValidatorContext implementations and ensuring untrusted user input is never concatenated directly into custom constraint violation templates (e.g., use parameterized message templates rather than raw string interpolation).
   d. **Attribution documenting how the vulnerability has been identified or documented:**
      i. **Source:** Cybersecurity and Infrastructure Security Agency (CISA), U.S. Civilian Government
      ii. **NVD Publish Date:** Jun 03, 2025
2. **Dependency:** log4j-api-2.12.1.jar
   a. **CVE ID:** CVE-2026-34480
   b. **Description:** Apache Log4j Core's XmlLayout https://logging.apache.org/log4j/2.x/manual/layouts.html#XmlLayout , in versions up to and including 2.25.3, fails to sanitize characters forbidden by the XML 1.0 specification https://www.w3.org/TR/xml/#charsets producing invalid XML output whenever a log message or MDC value contains such characters. The impact depends on the StAX implementation in use: * JRE built-in StAX: Forbidden characters are silently written to

the output, producing malformed XML. Conforming parsers must reject such documents with a fatal error, which may cause downstream log-processing systems to drop the affected records. * Alternative StAX implementations (e.g., Woodstox https://github.com/FasterXML/woodstox , a transitive dependency of the Jackson XML Dataformat module): An exception is thrown during the logging call, and the log event is never delivered to its intended appender, only to Log4j's internal status logger. Users are advised to upgrade to Apache Log4j Core 2.25.4, which corrects this issue by sanitizing forbidden characters before XML output.

c. **Solution:** Users are advised to upgrade to Apache Log4j Core 2.25.4, which corrects this issue by sanitizing forbidden characters before XML output.
d. **Attribution documenting how the vulnerability has been identified or documented:**
    i. **Source:** Apache Software Foundation
    ii. **NVD Publish Date:** Apr 10, 2026

5. Mitigation Plan

To address the security defects identified in the manual code review and static dependency analysis, Global Rain must implement a defense-in-depth remediation strategy for Artemis Financial. This plan synthesizes findings across application logic, third-party libraries, and operational practices, and aligns with Implementation Group 1 (IG1) of the CIS Critical Security Controls and modern DevSecOps standards. Remediation actions are prioritized by risk exposure, starting with immediate application-level code hardening and progressing to dependency remediation and build automation.

The first phase addresses vulnerabilities identified during the manual code inspection to eliminate exploitable defects in custom application logic. At the API boundary, refactor `CRUDController.java` by replacing the generic `@RequestMapping("/read")` annotation with an explicit `@GetMapping("/read")` mapping. This enforces HTTP verb constraints, prevents HTTP method tampering, and adheres to the principle of least privilege. In `GreetingController.java`, user input received via `@RequestParam` must be constrained using Bean Validation annotations (e.g., `@Pattern(regexp = "^[a-zA-Z\\s]+$")` and `@Size(max = 50)`) and must undergo contextual output escaping before model reflection, to mitigate reflected Cross-Site Scripting (XSS) risks.

Within the domain and data-access tiers, database credentials in `DocData.java` must be removed from the Java source code entirely. Database hostnames, credentials, and configuration settings must be externalized to environment variables or to an encrypted property vault managed by Spring Boot (`application.properties`). The database connection string must enforce transport-layer encryption by appending parameters such as `?useSSL=true&requireSSL=true&verifyServerCertificate`=true to prevent eavesdropping and Man-in-the-Middle (MitM) attacks. Also, database socket leakage must be prevented by wrapping the `Connection` instantiation in a Java try-with-resources statement to ensure connections close automatically. Diagnostic exposure must be remediated by replacing `e.printStackTrace()` with an enterprise SLF4J logger that sanitizes messages before logging and returns generic HTTP error responses to client endpoints. Finally, in `customer.java`, declare internal fields (`account_balance`) as `private` to restore encapsulation, and update state-modifying mutators to validate domain invariants (e.g., `deposit(int a)` must strictly reject negative or overflowing integer parameters).

The second phase resolves critical and high-severity vulnerabilities identified by the OWASP Dependency-Check scan, eliminating software supply chain risks in third-party artifacts. The parent Spring Boot starter in `pom.xml` must be upgraded to an active, supported release (such as Spring Boot 2.7.11+ or a modern 3.x baseline). This upgrade systematically resolves multiple critical flaws, including Spring4Shell (CVE-2022-22965) in `spring-webmvc` and `spring-core`, an authentication bypass via actuator routing (CVE-2023-20873), and the embedded Tomcat AJP connector vulnerability (Ghostcat, CVE-2020-1938). Upgrading the parent framework transitively pulls in patched versions of embedded Apache Tomcat (version 0.0.31+), which harden the default AJP configuration and neutralize arbitrary file-read and remote-execution vectors.

For standalone and transitive dependencies declared in `pom.xml`, enforce explicit version updates. Update SnakeYAML to version 2.0 or higher to eliminate arbitrary class deserialization (CVE-2022-1471), and instantiate any custom parsing logic using `new Yaml(new SafeConstructor())` to enforce strict type-instantiation boundaries. FasterXML Jackson Databind must be upgraded to version 2.10.5.1 or later to mitigate XML External Entity (XXE) vulnerabilities (CVE-2020-25649). In logging and validation frameworks, `logback-core` must be upgraded to version 1.2.13 or higher (CVE-2023-6378); `log4j-api` must be updated to version 2.25.4 or higher to prevent XML layout output truncation (CVE-2026-34480); and `hibernate-validator` must be updated to version 6.2.0.Final or later to disable default Expression Language interpolation in custom constraint messages (CVE-2025-35036). Bouncy Castle (`bcprov-jdk15on`) should be updated to version 1.56 or higher (or migrated to `bcprov=jdk18on`) to fix DSA signature encoding validation flaws (CVE-2016-1000338).

The third phase of the remediation strategy establishes DevSecOps integration and continuous verification to ensure that implemented security controls remain effective over time. Artemis Financial must institutionalize automated verification directly within the continuous integration and continuous delivery (CI/CD) pipeline. Permanently integrate Software Composition Analysis (SCA) via the OWASP Dependency-Check Maven plugin into the build lifecycle, and configure build-breaker thresholds to automatically halt compilation if unmitigated critical or high-severity CVEs are introduced. Concurrently, integrate Static Application Security Testing (SAST) linters, such as SonarQube, into developer IDEs and automated pipeline stages to flag code quality flaws, unclosed resources, and injection anti-patterns before merging code changes. By standardizing continuous dependency scanning, enforcing secure coding standards, and embedding automated pipeline guardrails, Artemis Financial will maintain ongoing compliance with the Gramm-Leach-Bliley Act (GLBA) Safeguards Rule and establish a robust, defensible software architecture (Center for Internet Security, 2024).

References

Center for Internet Security. (2022). *CIS Community Defense Model Version 2.0.* Center for Internet Security. Retrieved from https://www.cisecurity.org/insights/white-papers/cis-community-defense-model-2-0

Center for Internet Security. (2024). *A Cyber Defense Guide for the Financial Sector.* Center for Internet Security. Retrieved from https://www.cisecurity.org/insights/white-papers/a-cyber-defense-guide-for-the-financial-sector

Jeganathan, S. (2019). DevSecOps: A Systemic Approach for Secure Software Development. *ISSA Journal, 17*(11).